

# Lidar on an FPGA

...explained like you are five

Muhammad Refansa · FPGA study group · October 2026

This is the short, friendly version of the full guide. It has no register tables and almost no code. It answers one question: **what is going on inside this project, and why?** If you can follow this, you can follow the long guide, which has all the details, code and build steps. Every number here was measured on the real board or checked with a script, the same as in the long guide.

## 1. What did we build?

A **lidar** is a small machine that measures distances with a laser. Ours is an RPLIDAR A1. It spins around about 6 to 7 times every second. It fires its laser about 2000 times every second. Each time, it tells us: “In **this** direction, there is something **this** far away.”

### IMAGINE THIS

A lighthouse that spins and, instead of shining a beam for ships, uses a very fast measuring tape. It shouts “direction, distance!” two thousand times a second.

We connected it to a **Zybo Z7-10**, a small board with an **FPGA** chip on it. The board does four jobs:

1. **Wake the lidar up** and check that it is healthy.
2. **Spin its motor** at a speed we choose.
3. **Listen** to the stream of “direction, distance” messages and find where each full turn starts.
4. **Report**: print one line per turn, light some LEDs, and (if we want) send the whole picture to a PC that draws it.



Figure 1: The real setup. The lidar (bottom) is wired to the Zybo through a breadboard. Power for the lidar comes from a separate USB source, because the motor is hungry.

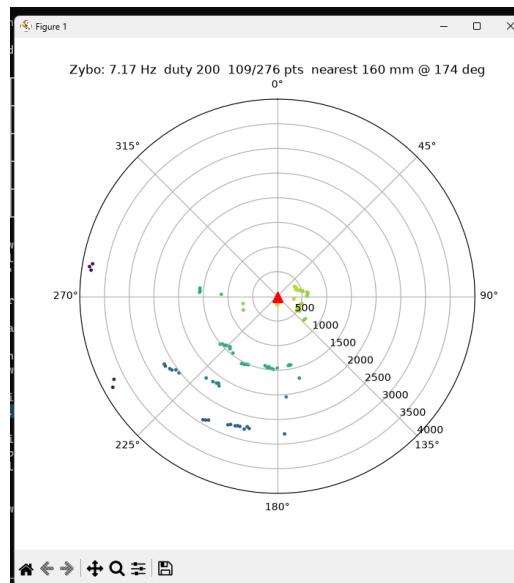


Figure 2: What the board hears, drawn by a PC. The red triangle is the lidar. Each dot is one “direction, distance” message. The rings show distance.

That picture is the whole point. Everything else in this guide is about how those dots travel from the spinning sensor into the board **without getting lost or mixed up**.

## 2. Meet the cast

### 2.1. The chip is a small house with two people in it

The big chip on the Zybo is called a **Zynq**. It has two parts, and it helps to think of them as two different people:

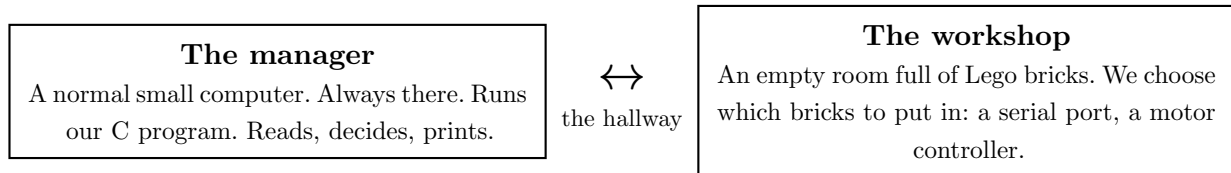


Figure 3: Inside the Zynq. The manager and the workshop talk through a hallway.

#### THE GROWN-UP WORD

The manager is the **PS** (processing system, an ARM processor). The workshop is the **PL** (programmable logic, the “FPGA” part). The hallway is called **AXI**.

## 2.2. Mailboxes with house numbers

How does the manager talk to a brick in the workshop? Every brick has a few **mailboxes**, and every mailbox has an address, like a house number. The manager writes a note into mailbox `0x41200000` and the motor controller changes its speed. The manager reads mailbox `0x42C00008` and learns whether a message has arrived. In C, this is only two actions: *read a mailbox* and *write a mailbox*.

#### THE GROWN-UP WORD

Mailboxes are called **registers**. This idea is called **memory-mapped** hardware.

## 2.3. The one-lane road: UART

The lidar sends its messages to the board over a **UART**. It is a very simple road: one wire, one lane, and the bytes go one after another. There is no clock wire. Instead, both sides agree on a speed before they start (here 115 200 bits per second). One byte takes 10 bits, so the road carries at most **11 520 bytes per second**, and not a single one more.

Two golden rules for this road: the sender’s TX wire goes to the receiver’s **RX** wire (a “talk” plug into a “listen” socket), and both sides must share a ground wire.

## 2.4. Flicking a switch very fast: PWM

How do you make a motor go “a bit slower” if you only have an on/off switch? You flick the switch on and off **very** fast (24 000 times a second). The motor cannot follow each flick. It just feels the average. If the switch is on half the time, the motor feels “half power”.

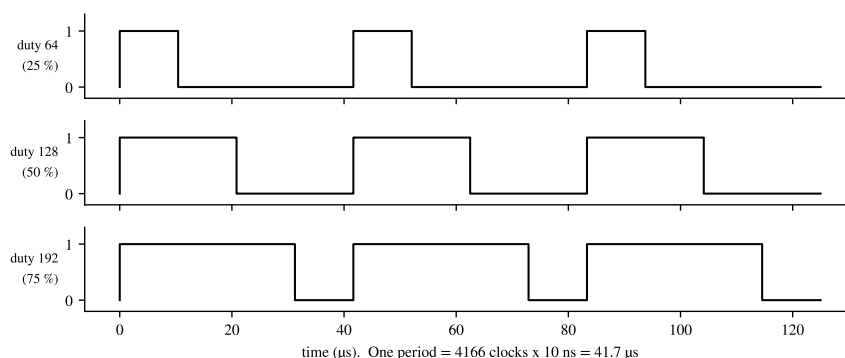


Figure 4: Three settings. The pattern repeats very fast. Only the fraction of time spent “on” changes.

This is the one thing we built ourselves in the workshop. It is a counter that counts around a loop, plus one question: “Is the counter below my limit? Then switch on.” About 15 lines of Verilog.

#### THE GROWN-UP WORD

This trick is **PWM** (pulse-width modulation). The fraction of “on” time is the **duty cycle**. Ours is an 8-bit number from 0 to 255. We use 120, which is on about 47% of the time.

### 2.5. A manager with no helpers: bare-metal

Our C program has no Windows or Linux under it. It is a single person doing a single endless loop: check this, check that, check this, check that. Nobody else can step in while it is busy. This one fact explains almost every problem we had, as you will see.

## 3. How the lidar talks

### 3.1. One message = one postcard

The lidar sends us tiny messages. Each is exactly **5 bytes**, like a postcard with a fixed set of boxes to fill in:

| Check marks            | Quality          | Direction              | Distance                  |
|------------------------|------------------|------------------------|---------------------------|
| “I am a real postcard” | “How sure am I?” | “Which way, 0 to 360°” | “How far, in millimetres” |

Figure 5: One postcard (a “node” in the long guide). Five bytes, four things to read.

About 1980 postcards arrive every second, forever, with **no gap and no marker** between them. They are just a long river of bytes.

### 3.2. Two small tricks that make the river readable

**Check marks.** Each postcard contains two special bits that are always opposites, plus one bit that is always 1. If a postcard breaks these rules, we know we are reading it wrong. It is like every postcard having a stamp in the corner: if the stamp is missing, we are holding it wrong.

**“New turn” flag.** The lidar never says “I finished a turn”. Instead, the first postcard of each new turn carries a flag. So one full turn is simply **everything between two flagged postcards**.

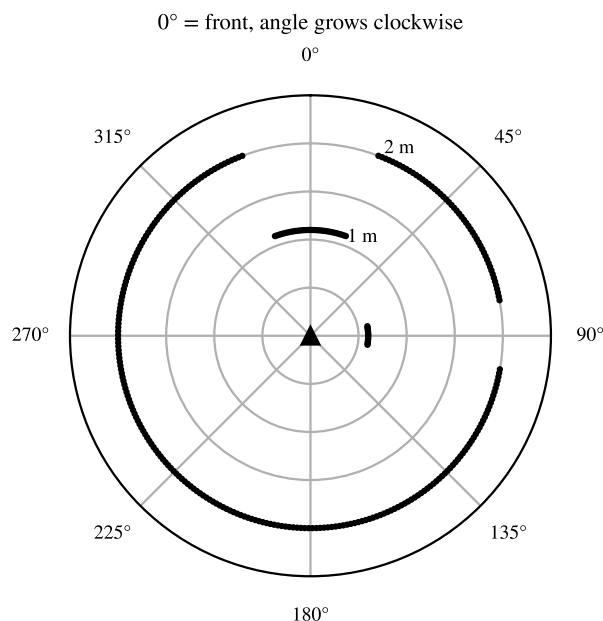


Figure 6: One turn, sorted by direction: a far wall, a close object to one side, something in front.

## 4. Not losing any mail

This is the part that surprised us most.

### 4.1. The tiny mailbox that overflows

The serial brick in the workshop has a mailbox that holds only **16 bytes**. The lidar fills it at about one byte every 100  $\mu$ s. How long until it is full? About **1.4 milliseconds**. If the manager does not empty it within that time, the next byte falls on the floor, and nobody tells us.

#### IMAGINE THIS

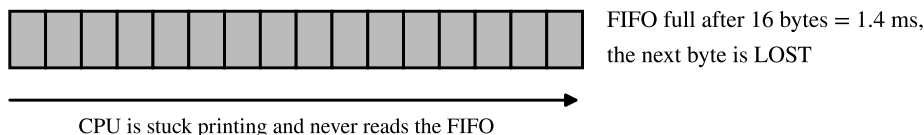
A mail slot that holds 16 letters, and a postman who delivers a new letter every tenth of a millisecond. If you leave to make a cup of tea, the floor fills with letters.

Now think about the manager **printing a line of text**. The normal print command waits for the screen's own slow road to take every letter. A 90-letter line takes about 8 milliseconds. During those 8 milliseconds, the manager is not looking at the mail slot. Over 60 letters arrive. Only 16 fit. Every single **print** would wreck a big part of a turn.

### 4.2. The fix: a notepad

We stopped printing directly. Now, when the program wants to print, it just **writes the text on a big notepad** (a 4 000-letter array in memory) and moves on instantly. Every pass around the loop, it also sends a few letters from the notepad to the screen, as many as the screen will accept right now, and moves on again. The mail slot is emptied on every pass.

#### Blocking print (xil\_printf)



#### Non-blocking ring buffer (this project)

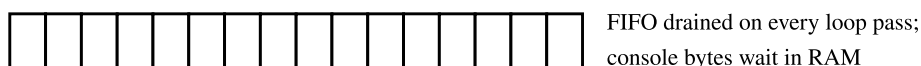


Figure 7: Top: a normal print stops the manager, so the mail slot overflows. Bottom: printing only writes to the notepad, so the mail slot is emptied on every pass.

The notepad is a **circular** one. A “write finger” and a “read finger” chase each other around it like runners on a circular track. When the write finger reaches the end, it jumps back to the start. And what if the notepad is full? Then the **screen** loses text, never the lidar. Losing a few letters of console text is harmless. Losing lidar data is not. In this project the sensor always matters more than the screen.

#### THE GROWN-UP WORD

The notepad is a **ring buffer** (or circular buffer). The mail slot is the UART's **FIFO**. A byte lost because the FIFO was full is an **overflow**.

### 4.3. Proof, not hope

The board counts lost bytes and prints the count on every line (*ovr*). In our run, it stayed at **0** for hundreds of turns. “It does not lose data” only counts if the program can show it.

## 5. Finding where a postcard starts

Suppose we start listening in the middle of a postcard, or one byte gets lost. Now every group of five bytes is shifted, and every “postcard” we read is nonsense.

### 5.1. Slide a window

The parser looks at the last five bytes through a little window and asks: “Do the check marks hold? Is the direction a real direction (below 360°)?” If not, it throws away **one** byte, slides the window along, and tries again. When the alignment is right, the check marks pass again and again.

### 5.2. One good window is not enough

How easily can **random noise** pass for a good postcard? We tested it on 200 000 random bytes:

| Check                            | Noise that fools it |
|----------------------------------|---------------------|
| Check marks only                 | 1 in 4              |
| Check marks and a real direction | 1 in 5.7            |
| ... three windows in a row       | about 1 in 180      |

Table 1: Measured with `check.py`.

One in four is far too weak: noise would fool us all the time. So the parser waits until **three** good postcards in a row line up before it believes the alignment. It also ignores any “turn” with fewer than 100 points. With these rules, 100 kB of pure noise gives **zero** fake turns.

#### IMAGINE THIS

Recognising a song. One note fits a thousand songs. Three notes in the right order fit almost none. The parser wants three notes before it says “I know this tune.”

### 5.3. Building one turn

Each accepted postcard goes into a table with **360 slots, one per degree**. If two postcards land in the same slot, we keep the **closer** one, because for safety the nearest obstacle matters most. When the next “new turn” flag arrives, the turn is finished. The first turn after start-up is always thrown away, because we joined it halfway through.

From the table the program gets one useful number: **how close is the nearest thing in front of us?** If it is closer than 1 metre, one LED turns on. If it is closer than 30 cm, another one does.

## 6. How we know it works

### 6.1. Never test two new things at once

If you connect two new things and nothing works, you cannot tell which one is broken. So we proved each half **on its own** first, then joined them one step at a time:

|   |                                                                                             |
|---|---------------------------------------------------------------------------------------------|
| 1 | Simulated the motor controller on the PC: the maths is right.                               |
| 2 | Fed the parser a fake stream on the PC: the parser logic is right.                          |
| 3 | Plugged the lidar into a laptop: the sensor works, and we understand it.                    |
| 4 | Connected the board's TX pin to its own RX pin (a loopback): the board's serial port works. |
| 5 | A fake lidar on the laptop, through the board: the board's software works.                  |
| 6 | The real lidar through the laptop, through the board: real data works.                      |
| 7 | The real lidar wired straight to the board: the whole system.                               |

Table 2: Seven steps. Each changes exactly one thing from the step before.

When step 7 first failed, steps 1 to 6 had already shown the board, the parser and the sensor were fine. So the problem had to be in the wires between them (it was, see below).

## 6.2. A watchdog that barks

What if a cable falls out while the motor is running? The program remembers the time of the last good postcard. If more than 1.5 seconds pass with none, it prints a message and restarts the whole start-up conversation with the lidar. It is like a guard dog: every good postcard feeds it, and if the food stops, it barks.

## 7. What we saw

With the lidar wired straight to the board, the console prints one line per turn:

```

COM5 - PuTTY
rev 405 7.07 Hz pts 280 valid 204 near 123 mm 0 17 deg front 123 mm duty 200 ovr 0 sync 15
rev 406 7.10 Hz pts 275 valid 204 near 123 mm 0 47 deg front 124 mm duty 200 ovr 0 sync 15
rev 407 7.09 Hz pts 275 valid 204 near 123 mm 0 48 deg front 124 mm duty 200 ovr 0 sync 15
rev 408 7.07 Hz pts 280 valid 195 near 123 mm 0 17 deg front 123 mm duty 200 ovr 0 sync 15
rev 409 7.09 Hz pts 279 valid 182 near 123 mm 0 15 deg front 123 mm duty 200 ovr 0 sync 15
rev 410 7.07 Hz pts 280 valid 180 near 124 mm 0 12 deg front 124 mm duty 200 ovr 0 sync 15
rev 411 7.07 Hz pts 280 valid 183 near 123 mm 0 39 deg front 123 mm duty 200 ovr 0 sync 15
rev 412 7.07 Hz pts 280 valid 204 near 123 mm 0 50 deg front 124 mm duty 200 ovr 0 sync 15
rev 413 7.04 Hz pts 281 valid 204 near 124 mm 0 49 deg front 125 mm duty 200 ovr 0 sync 15
rev 414 7.07 Hz pts 280 valid 203 near 123 mm 0 47 deg front 125 mm duty 200 ovr 0 sync 15
rev 415 7.07 Hz pts 280 valid 210 near 123 mm 0 46 deg front 126 mm duty 200 ovr 0 sync 15
rev 416 7.04 Hz pts 281 valid 208 near 124 mm 0 44 deg front 127 mm duty 200 ovr 0 sync 15
rev 417 7.07 Hz pts 280 valid 215 near 124 mm 0 35 deg front 128 mm duty 200 ovr 0 sync 15
rev 418 7.07 Hz pts 280 valid 210 near 125 mm 0 35 deg front 130 mm duty 200 ovr 0 sync 15
rev 419 7.07 Hz pts 280 valid 212 near 124 mm 0 147 deg front 129 mm duty 200 ovr 0 sync 15
rev 420 7.07 Hz pts 280 valid 214 near 125 mm 0 37 deg front 129 mm duty 200 ovr 0 sync 15
rev 421 7.04 Hz pts 281 valid 209 near 125 mm 0 36 deg front 128 mm duty 200 ovr 0 sync 15
rev 422 7.07 Hz pts 280 valid 209 near 124 mm 0 147 deg front 128 mm duty 200 ovr 0 sync 15
rev 423 7.07 Hz pts 280 valid 217 near 125 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 424 7.07 Hz pts 280 valid 215 near 124 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 425 7.07 Hz pts 280 valid 214 near 126 mm 0 37 deg front 129 mm duty 200 ovr 0 sync 15
rev 426 7.07 Hz pts 280 valid 215 near 125 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 427 7.09 Hz pts 275 valid 215 near 125 mm 0 36 deg front 129 mm duty 200 ovr 0 sync 15
rev 428 7.07 Hz pts 280 valid 208 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 429 7.07 Hz pts 280 valid 209 near 125 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 430 7.09 Hz pts 275 valid 213 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 431 7.07 Hz pts 280 valid 208 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 432 7.10 Hz pts 275 valid 214 near 124 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 433 7.09 Hz pts 275 valid 214 near 124 mm 0 36 deg front 128 mm duty 200 ovr 0 sync 15
rev 434 7.09 Hz pts 275 valid 208 near 125 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 435 7.07 Hz pts 280 valid 208 near 125 mm 0 37 deg front 128 mm duty 200 ovr 0 sync 15
rev 436 7.09 Hz pts 275 valid 210 near 124 mm 0 38 deg front 128 mm duty 200 ovr 0 sync 15
rev 437 7.07 Hz pts 280 valid 215 near 124 mm 0 38 deg front 128 mm duty 200 ovr 0 sync 15
rev 438 7.07 Hz pts 280 valid 214 near 124 mm 0 39 deg front 127 mm duty 200 ovr 0 sync 15
rev 439 7.09 Hz pts 275 valid 207 near 123 mm 0 38 deg front 127 mm duty 200 ovr 0 sync 15
rev 440 7.07 Hz pts 280 valid 210 near 123 mm 0 42 deg front 126 mm duty 200 ovr 0 sync 15
rev 441 7.07 Hz pts 280 valid 200 near 124 mm 0 15 deg front 124 mm duty 200 ovr 0 sync 15
rev 442 7.07 Hz pts 280 valid 193 near 123 mm 0 29 deg front 123 mm duty 200 ovr 0 sync 15
rev 443 7.07 Hz pts 280 valid 192 near 123 mm 0 16 deg front 123 mm duty 200 ovr 0 sync 15
rev 444 7.07 Hz pts 280 valid 172 near 123 mm 0 7 deg front 123 mm duty 200 ovr 0 sync 15
rev 445 7.07 Hz pts 276 valid 172 near 123 mm 0 56 deg front 124 mm duty 200 ovr 0 sync 20
rev 446 7.07 Hz pts 280 valid 167 near 123 mm 0 41 deg front 123 mm duty 200 ovr 0 sync 20
rev 447 7.07 Hz pts 280 valid 157 near 123 mm 0 353 deg front 123 mm duty 200 ovr 0 sync 20
rev 448 7.07 Hz pts 280 valid 161 near 124 mm 0 58 deg front 125 mm duty 200 ovr 0 sync 20
rev 449 7.09 Hz pts 275 valid 162 near 123 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 450 7.07 Hz pts 280 valid 163 near 123 mm 0 353 deg front 123 mm duty 200 ovr 0 sync 20
rev 451 7.10 Hz pts 275 valid 161 near 123 mm 0 58 deg front 125 mm duty 200 ovr 0 sync 20
rev 452 7.07 Hz pts 280 valid 163 near 123 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 453 7.09 Hz pts 275 valid 166 near 123 mm 0 58 deg front 123 mm duty 200 ovr 0 sync 20
rev 454 7.12 Hz pts 278 valid 161 near 123 mm 0 58 deg front 123 mm duty 200 ovr 0 sync 20
rev 455 7.12 Hz pts 278 valid 158 near 123 mm 0 59 deg front 123 mm duty 200 ovr 0 sync 20
rev 456 7.12 Hz pts 278 valid 161 near 123 mm 0 60 deg front 124 mm duty 200 ovr 0 sync 20
rev 457 7.12 Hz pts 278 valid 159 near 123 mm 0 60 deg front 123 mm duty 200 ovr 0 sync 20
rev 458 7.12 Hz pts 278 valid 161 near 124 mm 0 60 deg front 124 mm duty 200 ovr 0 sync 20
rev 459 7.15 Hz pts 277 valid 160 near 123 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 460 7.12 Hz pts 278 valid 162 near 123 mm 0 60 deg front 124 mm duty 200 ovr 0 sync 20
rev 461 7.12 Hz pts 278 valid 160 near 124 mm 0 59 deg front 124 mm duty 200 ovr 0 sync 20
rev 462 7.15 Hz pts 277 valid 159 near 123 mm 0 58 deg front 124 mm duty 200 ovr 0 sync 20
rev 463 7.12 Hz pts 278 valid 167 near 123 mm 0 355 deg front 123 mm duty 200 ovr 0 sync 20
rev 464 7.12 Hz pts 278 valid 156 near 123 mm 0 57 deg front 124 mm duty 200 ovr 0 sync 20
rev 465 7.15 Hz pts 277 valid 160 near 123 mm 0 55 deg front 124 mm duty 200 ovr 0 sync 20

```

Figure 8: Console output. About 7 turns a second, about 280 points per turn, and `ovr 0`: no lost bytes.

We changed the motor speed from slow to fast and wrote down what happened:

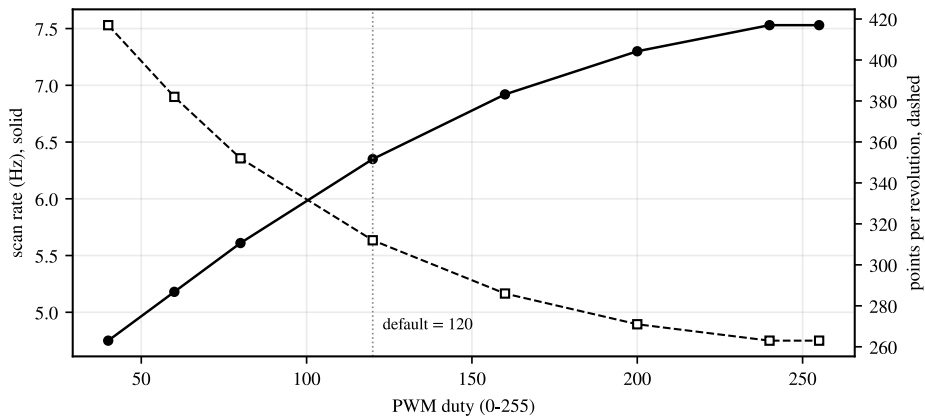


Figure 9: A faster motor gives more turns per second but fewer points in each turn. Above about duty 240 the motor cannot go faster.

The interesting part: **turns per second  $\times$  points per turn is always about 1980**. The lidar fires its laser at a fixed rate, whatever the motor does.

#### IMAGINE THIS

A sprinkler that always sprays the same number of drops per second. If you spin it faster, the drops spread over more circles, so each circle gets fewer drops. If you spin it slower, each circle is dense. You choose: **fresh picture** (fast) or **detailed picture** (slow).

We chose a middle setting: **6.35 turns per second** and **312 points per turn**, about  $1.15^\circ$  between neighbouring points.

One thing we did **not** prove: why the rate is exactly 1980. It could be the sensor’s own limit, or it could be the speed limit of the 115 200-baud road (at most about 2300 postcards per second), which is close. We say so rather than guess.

## 8. What went wrong

Real projects are mostly debugging. None of the problems below were mistakes in our clever code. They were timers, wires, paths and tools.

| What we saw                                        | What it really was                                                                                                                                                              |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The program froze at “Motor on, waiting 2 s”.      | The board’s timer only starts the first time something calls <code>usleep</code> . Nobody had, so “wait 2 seconds” never ended. Fix: one line at the top of <code>main</code> . |
| The board heard nothing from the lidar.            | Two outputs were wired to the same breadboard row. Two things shouting on one wire cancel each other out.                                                                       |
| Still silent, though the wiring matched our table. | Our colour table of the lidar cable had three signals on the wrong pins. The power wires were right. Lesson: check against the real connector’s printed labels.                 |
| Fake “turns” appeared out of noise.                | The 2-bit check passes noise 1 time in 4. Lesson: measure the false-alarm rate, then add the 3-in-a-row rule.                                                                   |
| The software tool could not build.                 | A space in the folder name. We built in <code>D:\lidar_build</code> instead.                                                                                                    |

Table 3: Five bugs and what they taught us.

The cure was always the same: **make the problem smaller until only one thing can be blamed.** That is exactly what the test ladder does.

## 9. Words you now know

|                       |                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------|
| <b>Lidar</b>          | A spinning laser tape measure. Reports (direction, distance) about 2000 times a second. |
| <b>Zynq PS / PL</b>   | The manager (an ARM computer) / the workshop (the FPGA part) of one chip.               |
| <b>AXI</b>            | The hallway between the manager and the workshop.                                       |
| <b>Register</b>       | A mailbox with an address that the manager can read or write.                           |
| <b>UART</b>           | A one-wire road for sending bytes at an agreed speed.                                   |
| <b>PWM</b>            | Flicking a switch very fast so a motor feels an “average” power.                        |
| <b>Bare-metal</b>     | Software with no operating system: one person, one endless loop.                        |
| <b>FIFO / overrun</b> | A small mail slot / a letter that fell on the floor because the slot was full.          |
| <b>Ring buffer</b>    | A circular notepad that lets slow printing and fast listening coexist.                  |
| <b>Parser</b>         | The code that finds where each postcard starts and builds a turn from them.             |
| <b>Watchdog</b>       | A timer that restarts things if the good news stops coming.                             |

## 10. Where to go next

If this made sense, the long guide ([Lidar\\_on\\_an\\_FPGA\\_Guide.pdf](#), same folder) goes deeper, in this order:

- **Chapters 1 and 2:** the same cast, with real addresses, a register table and the Zynq in detail.
- **Chapters 3 and 8:** the exact bits of a postcard and the parser, with a worked example you can decode by hand.
- **Chapter 4:** the Verilog motor controller, line by line.
- **Chapters 5 to 7:** the C program, the timer story, the ring buffer and the watchdog.
- **Chapter 10:** build and run everything yourself, step by step.
- **Chapter 13:** exercises, from pencil-and-paper to changing the hardware.

### IMAGINE THIS

The best habit in this whole project was not clever code. It was **testing each half alone before joining them.** Whenever something fails, ask: “Which half is the problem?”